HeatSheet Language Specification

Yanni Kakouris Asa Shepard

May 5, 2025

Introduction

Managing a complete roster of track and field athletes is both time-consuming and difficult for coaches. Each athlete has a preferred event, the event they're best at, and the event that they can score the most points in. Often, these are all different. For coaches, determining where athletes can perform best can be the difference between winning a meet and not. However, coaches tend to rely on intuition to determine the best event(s) for their athletes, which can be suboptimal in many cases.

Our language, HeatSheet, intends to streamline this process for coaches. By managing the roster under the hood, computing the expected points for an athlete in an event, and determining their comparative advantage, HeatSheet will complete all of the hard work for coaches. Given that many coaches aren't too technical or familiar with programming languages, our language will be designed to be as simple as possible. It won't require advanced syntax or programming knowledge to understand. Also, it will allow for coaches to overwrite our calculations if they strongly believe that an athlete should compete in a certain event (i.e. based on practice performance or gut feeling). The outputted text files will contain all the information a coach needs for a given athlete, roster, or competition.

Design Principles

HeatSheet will have a "coach-first" design. The language is built for track and field coaches, not programmers. Syntax and semantics prioritize readability, clarity, and domain familiarity. The language will be declarative, so coaches will be able to describe what they want, not how to do it step by step. HeatSheet will compute mostly under-the-hood, making it easy for coaches to quickly get the information they need. Athletes, rosters, and meets are modeled as composable objects, so users can easily adapt the same logic across different scenarios or competitions. The language will be designed to connect to a SQL backend that can efficiently and quickly handle the database for rosters, athletes, and meets. Ultimately, HeatSheet will be simple enough for dual meet planning, but powerful enough to handle championship meets or team selection for nationals if necessary.

Examples

Example 1:

```
let roster Williams
let athlete: YanniKakouris,
  events: 100m, 200m, 4x100m,
  prs:
    100m: 11.01,
    200m: 22.42
add YanniKakouris to Williams
```

Listing 1: HeatSheet Language Example 1

Example 1 defines a roster `Williams` and an athlete `YanniKakouris` with the specified events and personal records. It then adds `YanniKakouris` to the `Wililams` roster.

Example 2:

```
let meet: StateChampionships,
  events: 100m, 200m, 400m, 4x100m, 4x400m,
  scoring: 1st: 10, 2nd: 8, 3rd: 6, 4th: 5,
           5th: 4, 6th: 3, 7th: 2, 8th: 1,
  maxEventsPerAthlete: 4

optimize YanniKakouris for StateChampionships
optimize Williams for StateChampionships
```

Listing 2: HeatSheet Language Example 2

Example 2 defines a meet titled `StateChampionships` with the specified events, scoring system, and max events per athlete. It then optimizes `YanniKakouris` for this meet, placing him into the best possible events given his profile. Finally, it optimizes the entire `Williams` roster, placing each athlete into the best possible event to maximize score for `StateChampionships`.

Example 3:

```
let athlete: AsaShepard,
  events: 400m, 800m, 4x400m,
  prs:
    400m: 48.68,
    800m: 1:55.04
add AsaShepard to Williams

let athlete: AmherstMammoth,
  events: 400m, 800m,
  prs:
    400m: 49.00,
    800m: 1:56.00
add AmherstMammoth to Amherst

add Amherst to StateChampionships
force AsaShepard to 4x400m
force YanniKakouris to 100m

optimize Williams for StateChampionships

remove AsaShepard from Williams
remove YanniKakouris from Williams
```

Listing 3: HeatSheet Language Example 3

Example 3 defines another athlete, labeled `AsaShepard`. It then defines an athlete `AmherstMammoth` and places that athlete on the `Amherst` roster. It then includes them as a competitor at `StateChampionships`. Before optimizing the `Williams` roster given this new Amherst athlete, it forces `AsaShepard` and `YanniKakouris` into certain events. Finally, it removes `AsaShepard` and `YanniKakouris` from the `Williams` roster.

Language Concepts

HeatSheet is built on a set of domain-specific objects, all composed from a small number of primitive types. These primitives include strings, integers and floats, booleans, lists. These primitives form the foundation for the language's structured objects. The first is an athlete, a user-defined object that combines identifiers, eligible events, personal records, and other optional data. An event has a name, a type (individual or relay), and optional properties like scoring weight. A meet is a composite structure that bundles events together with a scoring system, global constraints, and scheduling rules. Coaches must understand what makes up each of these objects, which they already know from their experience coaching.

Coaches can impose constraints to model the realities of competition, such as limiting an athlete to four events or forcing an athlete into a certain event, even if it may not be theoretically optimal. These goals shape how the system evaluates possible lineups.

Formal Syntax

HeatSheet reads like a set of natural-language commands. A file is just a list of statements, one per line. It is enough for a coach to read, edit, and rerun without too much formal punctuation or brackets.

```
program              ::= <statement> ;+

statement            ::= <roster-declaration>
                       | <athlete-declaration>
                       | <meet-declaration>
                       | <meet-add>
                       | <roster-add>
                       | <output>
                       | <optimize>

<roster-declaration>    ::= let roster <Identifier>
                          | let roster <Identifier>,
                            athletes: <athlete-list>

<athlete-declaration>   ::= let athlete <Identifier>,
                             events: <event-list>
                          | let athlete <Identifier>,
                            events: <event-list>,
                            PRs: <pr-list>

<meet-declaration>      ::= let meet <Identifier>,
                             events: <event-list>,
                             scoring: <scoring-list>
                          | let meet <Identifier>,
                            events: <event-list>,
                            scoring: <scoring-list>,
                            teams: <roster-list>

<meet-add>              ::= include <roster-list> in <meet>

<roster-add>            ::= add <athlete> to <roster>

<output>                ::= output <roster>

<optimize>              ::= optimize <roster> for <meet>

<athlete>               ::= <Identifier>

<roster>                ::= <Identifier>

<meet>                  ::= <Identifier>

<athlete-list>          ::= <athlete> , <athlete>*

<roster-list>           ::= <roster> , <roster>*

<event-list>            ::= <event> , <event>*

<event>                 ::= <Identifier>
```

```
<pr-list>               ::= <pr> , <pr>*

<pr>                    ::= <event> : <time>

<scoring-list>          ::= <score-entry> , <score-entry>*

<score-entry>           ::= <place> : <int>

<place>                 ::= <int> <suffix>

<suffix>                ::= st | nd | rd | th

<time>                  ::= <float>
                          | <int> : <float>

<float>                 ::= <int> . <int>

<int>                   ::= <digit>+

<digit>                 ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

<Identifier>            ::= (<Letter> | <Digit>)*

<Letter>                ::= a-z | A-Z
```

Semantics

Primitive values include strings for names, integers, floats, time literals (stored internally as seconds with millisecond precision), booleans, and ordered lists. These values combine to form the four key objects: an *Athlete* (a name, eligible events, and a map from event to personal record), a *Roster* (a named collection of athletes), a *Meet* (its events, scoring table, and global constraints such as the maximum events per athlete), and an *Assignment rule* generated by a `force` statement. All objects are immutable once declared. Only the references inside rosters and meets change when coaches add or remove athletes.

Evaluation proceeds top-to-bottom. The interpreter builds an in-memory catalogue as it reads. Each `add`, `remove`, or `force` mutates that catalogue in place. When it encounters `optimize Roster for Meet`, HeatSheet computes expected points from personal records, searches the legal assignment space while respecting every explicit `force`, and selects the best lineup. Results are written to a plain-text file named `<roster>-<meet>-lineup.txt` and echoed to the console; there are no other side-effects. An unknown identifier aborts execution with a clear error message, whereas non-fatal issues such as a missing personal record only generate warnings.